



zodal-groups: Architecture and Design Rationale for Item–Group Membership — Folders, Tags, Taxonomies, and Facets

Version 0.1 · Companion to the [research corpus](#) (five reports, ~264 cited sources) and the [reconciliation](#) (24 numbered decisions, D1–D24).

1. Abstract

Applications that organize things converge, early and irreversibly, on one of four schemes — folders, tags, categories, or facets — and inherit that scheme's limitations forever. This document argues that these are not four data models but **four projections of one model**, differing only in which constraints they impose, and that the near-universal "an item lives in exactly one place" restriction is a property of a *rendering*, not of the data. A survey of the standards literature (ANSI/NISO Z39.19, SKOS, OWL 2, the Gene Ontology), of production systems (Gmail, Zotero, MeSH, [Are.na](#), Google Drive, Danbooru), of hierarchy-storage encodings (adjacency list, materialized path, nested set, closure table), and of the JS/TS library landscape establishes that the model is well understood in theory, routinely mis-implemented in practice, and — critically — **not available anywhere as a constraint-parameterized, projection-aware library**.

`zodal-groups` occupies that gap. It stores a single canonical relation of reified membership edges and computes every tree, tag cloud, breadcrumb, and facet panel as a pure projection over it; the choice between "filesystem" and "tag cloud" becomes a runtime dial (`maxParentsPerItem`) rather than an architectural commitment. The cost of this posture is a hard prohibition on cycles and a refusal to expose a `node.parent` accessor — two constraints we accept deliberately, and defend below.

2. The Problem Landscape

2.1 The core problem

Any system with more than a few hundred items needs a way to group them. The available schemes trade against each other along axes that are rarely made explicit:

Scheme	Nesting	Multi-membership	Group-of-groups	Typical failure
Folders	✓	✗	✓	"Where did I file that?" — a document that is <i>both</i> a contract <i>and</i> a Q3 record has to pick one.
Flat tags	✗	✓	✗	Tag sprawl. No way to say <code>poodle</code> implies <code>dog</code> .
Categories	✓	✗ (items attach at one node)	✓	The taxonomy is expressive; the item's relationship to it is not.
Facets	✓ (per facet)	✓	✓	Usually bolted onto search, disconnected from the browse hierarchy.

The folder model is the one most systems adopt, and its defining restriction — one item, one location — is so pervasive that it is generally taken for a law of nature. It is not. **POSIX filesystems already violate it:** a file may be hard-linked into arbitrarily many directories, and each directory entry is an equal, first-class name for the same inode [1]. What POSIX forbids is hard links *to directories*, and it forbids them for one specific, non-obvious reason: to keep the directory graph acyclic, so that reference-count garbage collection terminates [1, 2].

That is not a prohibition on multi-membership. It is a **constraint profile**:

```
maxParents(file)      = ∞
maxParents(directory) = 1
allowCycles            = false
```

The insight that organizes this entire package follows directly: *"one place" is a dial, not a law*. The four schemes above are the same relation under four settings of that dial, plus a choice of how to draw it.

2.2 Hidden complexities

The moment the single-parent restriction is lifted, a set of problems appears that has no analogue in the tree case. These are the substance of the design, and each is a place where an intuitive implementation is wrong.

(a) A node is not a row. In a tree, a node appears once, so a node *is* a row and its identity can serve as a render key. In a directed acyclic graph (DAG), a node appears once *per path to it*. If group `G` sits under both `A` and `B`, a tree view legitimately shows it twice — and the user can open one occurrence and leave the other closed. Any implementation that keys UI state by node identity will therefore exhibit one of two bugs: expanding a node causes it to expand *somewhere off-screen* (if expansion is node-keyed), or selecting an item leaves its other appearances un-highlighted, so one thing looks like two (if selection is path-keyed). Both survive code review.

(b) Transitivity is not free, and blind transitivity lies. The natural reading of "everything under `animal`" is a transitive closure. But closure over heterogeneous edges produces false conclusions:

A wheel is part_of a car. A car is_a a vehicle. A wheel is not a vehicle.

SKOS makes `skos:broader` deliberately **non-transitive** for exactly this reason and provides a separate `skos:broaderTransitive` for when the inference is warranted [3]. The Gene Ontology earns transitivity only by declaring explicit relation-composition rules (`is_a ◦ part_of → part_of`) and *excluding* unsafe combinations [4]. Z39.19 gates the broader/narrower relation behind an "all-and-some" test and instructs the vocabulary designer to **demote a multi-parent whole-part edge to an associative relation** rather than pretend the hierarchy is sound [5]. Consequently, `poodle` $\diamond$ `animal` **is not answerable from the edge set alone** — only from the edge set *plus* declared per-kind closure semantics.

(c) The diamond problem. If an item is in `A`, `A` is under both `B` and `C`, and `B` and `C` are both under `D`, then `D` is reachable twice. An un-memoized traversal is $O(2^n)$; an un-deduplicated aggregate double-counts. This is not hypothetical: Solr's documented default for multivalued fields is that a document with two values "will be counted in each bucket" [6].

`Σ children.count` becomes wrong the instant the structure stops being a tree — and *wrong*, not *broken*, which is far worse, because a plausible number is not investigated.

(d) Paths stop being identifiers. `/usr/local/bin` works as a name because there is exactly one route to it. Under polyhierarchy a path is a **route**, not an identity, and the number of distinct root-paths is exponential in depth (a chain of d diamonds yields 2^d). Breadcrumbs, URLs, and deep links all need an explicit, *visible* choice of one route. MeSH is the cautionary tale: because its parent edges live between *tree-number positions* rather than between concepts, its descriptor-level `broaderDescriptor` shortcut disagrees with the tree walk — a discrepancy its own documentation warns about [7].

(e) Cycles are real, and the user cannot see them. Cycle prevention in a tree is visually obvious — you cannot drag a folder into itself, and the illegal targets are the ones inside the thing you are dragging. In a DAG the loop can close through a branch that is **not on screen**. A drop target that simply refuses, with no explanation, is therefore *indistinguishable from a bug*. Worse, real taxonomies contain cycles: a study of the Open Directory Project found its symbolic links "by inducing cycles, preclude the underlying graph model from being a DAG" [8], and Wikipedia's category graph is famously cyclic [9].

(f) `remove` $\neq$ `delete`, and the last group is a special case. Taking an item out of one group must not destroy it. Every serious product makes this distinction (Gmail's remove-label vs. delete; Lightroom's "Remove from Collection" vs. "Delete from Disk" [10]), and each must then answer: what happens when the item leaves its *last* group? Three answers exist in the wild — an implicit universal group (Gmail's All Mail), an explicit orphan view (Zotero's **Unfiled Items** [11]), or refusal.

(g) Drag-and-drop becomes genuinely ambiguous. In a tree, dragging means *move*. In a DAG, it could mean *move* (remove the old edge) or *add a parent* (keep both). The two look identical mid-drag and differ in whether data disappears — and *move* is literally **undefined** when the drag originates in a search result or a flat "all items" list, because there is no source group to remove from.

(h) Ordering is per-membership, not per-item. An item in three groups needs three ranks. A

`position` column on the item is structurally incapable of expressing this.

(i) **The presentation problem is not the modelling problem.** Google Drive **abolished multi-parenting** in September 2020 — "it is no longer possible to place an item in multiple folders" — migrating extra parents to visible *shortcut* nodes and adding an `enforceSingleParent` API flag [12, 13]. This is the strongest empirical signal in the entire survey, and it must be read correctly: it is not evidence that the model is wrong, it is evidence that a *naked DAG rendered as a folder tree* is unteachable. Hearst says the same thing directly — do not teach polyhierarchy through a folder metaphor, because "users would be unfamiliar with the idea of an item simultaneously residing in multiple folders" [14]. Same information; different projection.

3. Prior Work & Competitive Analysis

3.1 Standards and formal models

Source	What it establishes	Limitation for our purposes
ANSI/ NISO Z39.19 [5]	The vocabulary ladder — $\text{list} \subset \text{synonym ring} \subset \text{taxonomy} \subset \text{thesaurus}$ — presented explicitly as increasing structural commitment. Polyhierarchy is sanctioned. BT/NT split into BTG/BTP/BTI (generic/partitive/instance).	Prescriptive for <i>human-authored</i> vocabularies; silent on storage, UI, and dynamic editing.
SKOS [3]	<code>broader / narrower</code> (non-transitive) vs. <code>broaderTransitive</code> ; <code>related</code> disjoint from <code>broaderTransitive</code> (S27); <code>Concept</code> disjoint from <code>Collection</code> (S37). Multiple <code>broader</code> is <i>permitted</i> .	A data-interchange vocabulary, not an implementation. Provides no cardinality constraints and no projection model.
OWL 2	The framing we borrow wholesale: <i>one</i>	About description-logic

Source	What it establishes	Limitation for our purposes
Profiles [15]	<i>model, several named syntactic restrictions, each buying a computational guarantee.</i>	tractability, not grouping.
SHACL [16]	A constraint language over graphs — <code>sh:maxCount</code> on a <code>broader</code> path is literally our <code>maxParentsPerGroup</code> ; validation yields a structured report.	Validation only. No notion of projection, closure caching, or UI. Closest existing constraint language; still not a grouping library.
Gene Ontology [4]	Transitivity earned via declared relation composition; the "true path rule".	Domain-specific; a curated artifact, not a runtime.
Datalog (EDB/IDB) [17]	The precise names for asserted vs. derived membership — extensional vs. intensional .	A query paradigm; the full recursive machinery is far beyond what a grouping UI needs.
Formal Concept Analysis	Concept lattices over object–attribute incidence.	Honestly assessed: a trap . Concept counts are exponential, lattices are unstable under small edits, and it has no notion of relation kinds. We retain only the implication basis as an authoring hint.

3.2 Production systems

The decisive split is **path-as-string vs. real edges**.

System	Model	Verdict
Gmail labels [18]	Labels are multi-membership; the <code>Label</code> API resource has no parent field — <code>"a/b/c"</code> is one string.	The canonical "folders that aren't folders". But nesting is a <i>naming convention</i> : polyhierarchy of labels is

System	Model	Verdict
		unrepresentable, rename is $O(\text{items})$, and there is nowhere to hang edge metadata. Its two-verb UI (<code>Label</code> vs <code>Move to</code>) is nonetheless the best answer anyone has to problem (g).
Obsidian / Bear / Logseq [19]	Nested tags as <code>#parent/child</code> strings.	Same limitation, and Logseq's namespace queries silently do not traverse — the visible crack in the string model.
Zotero [11]	Collections (multi-membership) + tags. <code>View → Show Items from Subcollections</code> is a user-toggleable view flag ; "Unfiled Items" is a computed group.	The best single piece of evidence for the thesis in existence: the same edges render as strict-containment or closure-expanded depending on a checkbox. The hierarchy is <i>literally</i> a projection.
Are.na [20]	Blocks live in unlimited channels; channels are blocks; "connections" are reified edges; "This channel appears in" is a first-class reverse index.	The closest existing product to our model. Validates <code>groupsAreItems</code> and <code>otherLocations</code> .
MeSH [7]	A large production polyhierarchy via tree numbers.	Demonstrates polyhierarchy at scale — <i>and</i> the shortcut-vs-tree-walk drift that motivates our <code>redundantEdge</code> lint.

System	Model	Verdict
Danbooru [21]	Tag implications with write-time closure.	The one production write-time-closure system. It needed three validators (no cycles, no chains, no redundancy) and still cannot cleanly <i>un-apply</i> an implication. A cautionary data point for D9.
Google Drive [12, 13]	Abolished multi-parenting (2020); extra parents → shortcuts.	See §2.2(i). A verdict on <i>presentation</i> , not on the model.
Unix VFS [1, 2]	Hard links: one inode, many dentries. Names live on the dentry, not the inode. Directory hard links forbidden.	The original polyhierarchy, and the origin of our names-on-the-edge and acyclicity decisions.
Git [22]	Merkle DAG: blobs shared across trees; the filename lives in the tree <i>entry</i> , not the blob; literals are content-addressed.	The model for `Member = ref
Zanzibar [23]	Flat relation tuples where the child may itself be a group — our model exactly. Required a separate denormalized transitive-closure index (Leopard) kept fresh by a watch stream.	Industrial validation <i>and</i> warning: the closure is a projection with a change-feed, not a property of the relation.

3.3 Storage encodings

The literature's four canonical answers, evaluated on the axis that actually matters here.

Encoding	Multi-parent?	Re-parent cost	Verdict
Edge table (bridging /	✅ Native	O(1) — one row	✅ The canonical store.

Encoding	Multi-parent?	Re-parent cost	Verdict
adjacency)			
Closure table	✓ Native (precomputes all overlapping paths)	Expensive; deletion requires a <code>path_count</code> refcount — the DRed problem [24]	✓ But only as a <i>derived cache</i> of the small graph.
Recursive CTE over adjacency [25]	✓ Native	O(1)	✓ Fine; cost is paid on read.
Materialized path / ltree [26]	✗ Fails combinatorially	Very expensive	✗ Never canonical.
Nested set (Celko) [27]	✗ Structurally impossible	Very expensive	✗ Never, at all.

Two findings deserve emphasis, because both are counter-intuitive and both are traps:

- **Nested set is not merely slow — it is structurally incapable of multi-parent.** It encodes containment in a *linear order*, so one interval = one position = one parent. It is nonetheless **the encoding most likely to surface from a naive search**, precisely because it optimizes the one metric everyone benchmarks first (subtree read).
- **Materialized path fails combinatorially, not linearly.** Forcing multi-parent means storing a *path set* per node, and distinct root-paths in a DAG are exponential in depth. Adding one edge high in the graph multiplies every descendant's path count. (Postgres's `ltree[]` GiST opclass is additionally, explicitly, **lossy** [26].)

3.4 Faceted search engines

Algolia's `hierarchicalMenu` widget and its `lvl0` / `lvl1` / `lvl2` attribute convention [28] are the only native multi-parent hierarchical-facet encoding in wide production use — and it works for Algolia precisely because their taxonomy is *authored and re-indexed*, which is the assumption we are voiding. Copying it would mean that a drag-and-drop in the group tree

rewrites every item beneath it. Solr's `excludeTags` [6] and Meilisearch's facet distribution [29] independently confirm that **correct disjunctive facet counts require N+1 queries** — one per selected facet, with that facet's own filter removed. This is architectural, not a feature flag.

3.5 The JS/TS library landscape

Surface	State of the art	Gap
Tree state	<code>headless-tree</code> , TanStack Table sub-rows, Zag/Ark <code>TreeView</code> , MUI X, rc-tree	Every one keys UI state by node id — so a node under two parents expands in both. The ones that let you <i>supply</i> identity (<code>getChildren</code> , <code>nodeToValue</code> , <code>getRowId</code>) become DAG-capable only if fed synthetic path ids . Nobody computes those.
Drag & drop	<code>pragmatic-drag-and-drop</code> (Atlassian)  ; <code>dnd-kit</code> frozen since Dec 2024 despite 16M weekly downloads; <code>react-dnd</code> dead since 2022	The download counts are a trap.
Miller columns	—	The category is a graveyard. No maintained library.
Space-filling viz	<code>d3-hierarchy</code>	<code>d3.hierarchy</code> is <i>strictly a tree</i> . No correct space-filling treemap of a DAG exists.
Containment graph	ELK, Cytoscape compound nodes	ELK is EPL-2.0, 423 kB.
Polyhierarchy per se	—	npm's entire inventory for "polyhierarchy" is one two-week-old GPL widget with 9 downloads/week . Its inventory for "transitive closure" is <i>the Google Closure Compiler</i> .

3.6 The gap, stated precisely

Every piece exists somewhere. SHACL can express the constraints; Z39.19 describes the ladder; OWL 2 establishes the "profile" framing; Zotero ships the projection idea as a checkbox; [Are.na](#) ships the edge model; Zanzibar ships it at scale. **But no library lets you write**

```
defineGroups({ profile: 'polyhierarchy', overrides: { maxDepth: 3, maxGroupsPerItem: 5 } })
```

and get a validated, projection-aware, type-safe grouping model out. Every tree builder, every ORM tree plugin (including those with a closure-table mode), and every graph renderer's containment model is strictly single-parent. That is the whitespace `zoda1-groups` occupies.

4. Design Principles & Dimensions

4.1 The dimensions of the problem

Making these axes explicit is what turns four schemes into one model with dials.

Dimension	Poles	Our position
Cardinality (item)	one parent ↔ many	A dial (<code>maxParentsPerItem</code>). Default <code>∞</code> .
Cardinality (group)	one parent ↔ many	A dial (<code>maxParentsPerGroup</code>). <code>1</code> ⇒ the group graph is a forest.
Nesting	flat ↔ arbitrary depth	A dial (<code>maxDepth</code>). <code>0</code> ⇒ flat tagging.
Homogeneity	items and groups distinct ↔ unified	Unified. Group-ness is <i>having children</i> , not a type. Bipartite-ness is a profile predicate.
Extension	extensional (member list) ↔ intensional (predicate)	Both , using Datalog's names. Intensional groups are leaf-only in v1.

Dimension	Poles	Our position
Closure	write-time (materialize) ↔ read-time (expand)	Neither, exactly — see §5.3. The dichotomy is false.
Truth location	the hierarchy is stored ↔ the hierarchy is computed	Computed. The relation is stored; every hierarchy is a projection.
Cycles	permitted ↔ forbidden	Forbidden on write; tolerated on read. See §5.5.
Reactivity	framework-coupled ↔ pure	Pure. No reactivity library in the core.

4.2 Foundational principles

1. **Membership is the canonical data.** A flat set of reified edges. Nothing else is authoritative.
 2. **Everything else is a projection.** Trees, tag clouds, breadcrumbs, facet panels, and counts are *pure functions* of (edges, profile, options) .
 3. **One model, named restrictions, each buying a guarantee.** (Borrowed verbatim from OWL 2 Profiles [15].)
 4. **Honest capability reporting.** An adapter states what it can do natively — and, specifically, *what happens on delete* — rather than exposing a boolean that means nothing.
 5. **Headless first.** The core emits configuration objects, never DOM.
 6. **Progressive disclosure.** A flatTags user should never encounter the word "polyhierarchy".
 7. **Acyclic on write; cycle-safe on read.** We enforce the invariant, and never *trust* it.
 8. **Don't reinvent the wheel — but own the gap.** Adapters wrap existing tools. The path-keyed DAG unfolding is the one thing no library provides, so the core owns it.
-

5. Architectural Choices & Trade-offs

Each subsection traces a design decision back to a specific problem in §2 or a specific gap in §3, and names the module that implements it.

5.1 One reified edge relation, two indexes

Answers §2.1 (the four-schemes trap) and the `groups_to_tags` / `tags_to_groups` synchronization problem.

Choice. `GroupSpace` (`groups-core/src/model.ts`) holds `nodes`, `edges`, and two indexes: `forward` (`group → members` — the *groups* view) and `inverse` (`node → groups` — the *tags* view). Both are maintained **in the same operation by the same writer** (`applyDelta`, `groups-core/src/space.ts`).

Rationale. The naive design is two structures — a folders tree and a tags map — kept in sync by an observer. That *is* index drift, waiting. Three independent systems converged on the same canonical form (RDF triples, Datomic datoms [30], Zanzibar relation tuples [23]), and Datomic in particular maintains **AVET** and **VAET** — a forward and a reverse covering index — for precisely our reason. Boost.MultiIndex frames it exactly right: "indices act as views to the internal collection" [31].

Trade-off. We sacrifice the ability to store a hierarchy directly (there is no tree to inspect in a debugger) in exchange for making index drift **unrepresentable rather than merely unlikely**. Both views are always live; there is never a conversion step.

Explicitly not CQRS. CQRS's defining property is *eventual* consistency between write and read models [32]; ours is synchronous. The label would invite the wrong mental model — and Fowler's own warning about it.

5.2 The edge carries `kind`; closure semantics belong to the kind

Answers §2.2(b) — blind transitivity lies.

Choice. `EdgeKindDef` (`groups-core/src/model.ts`) declares `{ transitive, symmetric, acyclic, composesWith, disjointWith }`.

`DEFAULT_EDGE_KINDS` ships `contains`, `is_a`, `part_of`, `instance_of`, `related`. The closure walk (`groups-core/src/closure.ts`, function `compose`) consults these at **every hop** and stops where the chain does not compose.

Rationale. Directly from SKOS's non-transitive `broader` [3], GO's relation composition [4], and Z39.19's BTG/BTP/BTI split [5]. Without it, `wheel part_of car is_a vehicle` concludes that a wheel is a vehicle.

Trade-off. We accept a more complex closure implementation, and a conservative default (`absent composesWith`, **a chain only continues through the same kind**) that will occasionally under-infer. We prefer under-inference to a system that confidently returns wrong answers. This is the single largest thing most tagging libraries omit, and it is the difference between a toy and a thesaurus.

Verification: `groups-core/tests/polyhierarchy.test.ts` — *"does NOT conclude that a wheel is a vehicle"*.

5.3 Closure of the *group DAG* only — the false dichotomy dissolved

Answers §2.2(c) and the write-time-vs-read-time question that §3.3 and §3.4 both pose.

The reframing. Every prior treatment conflates two graphs that have nothing in common but their edges:

	the group DAG	the membership relation
size	tiny — 10^2 – 10^4 nodes	huge — 10^6 + rows
change rate	rare (an admin re-parents a folder)	constant

Choice. Materialize the closure of the **group DAG only**; never of the items. The edge table is the source of truth (re-parent = **O(1)**); the group closure is small enough to *rebuild* rather than incrementally maintain; direct memberships are denormalized onto items as an indexed set. A read is: expand group → descendant group ids → `arrayContainsAny`. Implemented as `closureIds()` and `scopeFilter()` (`groups-core/src/closure.ts`, `groups-core/src/projections/facets.ts`).

Consequences, and they are large:

- **The update storm is structurally absent.** No item row is ever touched when the taxonomy changes. This is what sinks the Algolia `lvl0/lvl1/lvl2` encoding [28] for a *dynamic* hierarchy: a drag-and-drop would rewrite every item beneath the moved node.
- **The `path_count` / DRed deletion problem [24] evaporates.** We rebuild a small table rather than incrementally maintaining a large one. (The problem is real and subtle: deleting edge `3→4` must not delete closure row `(1,4)` if `1→2→4` still exists — a reference count, sound only under acyclicity, which is *the same fact* as Unix forbidding hard-linked directories.)
- **No new filter operator is needed.** `arrayContainsAny` already exists in `@zodal/core` and maps to Postgres `&&`, PostgREST `ov`, and Dexie `anyOf`. An earlier plan added a `descendantOf` operator; it was **abandoned as unnecessary**, which is a strictly better outcome.

Trade-off. We sacrifice single-probe reads for arbitrarily deep *item*-level queries (we always expand a group id set first) in exchange for O(1) writes and zero closure invalidation. The asymmetry in the table above is what makes this the right side of the trade — and if a future workload inverts that asymmetry, the decision should be revisited.

5.4 PathNode[] — the universal projection output

Answers §2.2(a) — a node is not a row — and the library gap in §3.5.

Choice. `projectTree()` (`groups-core/src/projections/tree.ts`) unfolds the DAG into a **flat, ordered array of path-nodes**, each carrying both identities:

```
interface PathNode {
  nodeId: NodeId;           // MODEL identity → selection, membership ops, cross-highlight
  pathKey: string;          // VIEW identity → expansion, DOM key, virtualization, ARIA
  path: readonly NodeId[];
  depth: number;            // → aria-level
  otherParentCount: number; // → "also in 2 other groups"
  isRecursive: boolean;     // cycle guard — NOT optional
}
```

The keying rule, and why it is not a compromise:

state	keyed by	because
expansion	pathKey	It is a <i>view</i> fact — "I opened this drawer", and there really are two drawers.
selection	nodeId	It is a <i>model</i> fact — "I chose this thing", and there is only one thing.
DOM key	pathKey	Duplicate DOM ids are invalid HTML and silently corrupt <code>aria-owns</code> and label associations.

Rationale — three independent forces converge on the same structure:

1. **ARIA.** `aria-owns` explicitly forbids multiple owners — *"Do not specify the id of an element in more than one other element's `aria-owns` attribute"* [33]. **The accessibility tree is a tree, by construction.** So the DAG must be unfolded into path-nodes *before* it reaches the DOM; then `aria-level = depth + 1` is unambiguous, *because the path is what got you here*. This is not a workaround — it is the correct reading of the spec.
2. **Virtualization.** Every virtualized tree flattens to visible rows. That flattened array is `PathNode[]`.
3. **The library gap.** No tree library computes a path-keyed unfolding; every one can render a flat list. Feeding a library synthetic path ids (`getRowId` , `nodeToValue` , `getChildren`) makes it DAG-capable. **This is the adapter for the entire ecosystem**, and it is why the core must own the projection.

Multi-parenthood is then conveyed **semantically, never structurally**:

`groups-ui/src/views.ts` composes the accessible name *"Reading, tree item, level 3, also in 2 other groups"*, and `otherLocations()` (`groups-core/src/projections/paths.ts`) provides a flat, linear, fully-navigable list of the other parents — which is strictly better than trying to force a tree to express two owners.

Trade-off. `PathNode[]` can be larger than the node set (a node appears once per visible path). We bound this by making the projection **O(visible rows), not O(graph)** — only expanded rows contribute children — and by capping `allPaths()` at 32 routes by default, because path enumeration is exponential in depth (§2.2(d)).

Verification: `groups-ui-vanilla/tests/render.test.ts` — *"keys DOM elements by PATH, so the same node twice is still valid HTML"*.

5.5 Acyclic on write; cycle-safe on read

Answers §2.2(e).

Choice. `applyDelta()` refuses any delta that would close a cycle, and the resulting `Violation` carries the offending path (`groups-core/src/space.ts` , `findCycle`). There is no `allowCycles` flag. Simultaneously, every projection is cycle-safe: `PathNode.isRecursive` is mandatory, and every traversal memoizes and guards (`groups-core/src/closure.ts`).

Rationale. Cycles destroy closure, recounting, and termination — the same reason Unix forbids hard-linked directories [1, 2]. But `zodal-groups` does not own its data: edges arrive from store adapters, imports, and foreign systems that never heard of our invariant, and real taxonomies *do* contain cycles [8, 9]. A projection that assumes acyclicity is a projection that hangs the browser on someone else's bad data. Enforce on write; never trust on read.

Why the path, not a boolean. Under polyhierarchy the cycle can close through an off-screen branch, so a bare refusal is indistinguishable from a bug. `canAddTo()` returns the violations; `groups-ui/src/drag.ts` renders them as *"That would create a loop: Reading → Research → Archive → Reading."* **Without that sentence, correct cycle prevention looks broken.**

Trade-off. Users who genuinely need cyclic structure are turned away — correctly. They need a graph library (`zodal-graphs`), not this one. `zodal-groups` is the *acyclic, containment-shaped special case*, and it is a special case **precisely so that closure, counts, and breadcrumbs can be well-defined.**

5.6 Unified node type; profiles as runtime validators

Answers §2.1 (one model, four schemes) and the gap in §3.6.

Choice. An item and a group are the same type; group-ness is *having children* (`isGroup()` in `groups-core/src/space.ts`). `NodeId` is branded; `ItemId` / `GroupId` are **not** — an item may be a group. Bipartite-ness (Gmail: a label is never a message) is a **profile predicate**, not a type distinction. `GroupProfile` and the eight named presets live in `groups-core/src/profile.ts` ; enforcement is in `validateEdge()` .

Rationale. With a unified node, every constraint profile becomes a one-line predicate over one relation (`groupsMayContainGroups === false` $\diamond \forall e: \neg \text{isGroup}(e.\text{child})$). With distinct

types, "filesystem" (where a directory *is* a file, per Unix and Git [1, 22]) cannot be expressed without reintroducing a union — and [Are.na](#)'s channel-as-block [20] is foreclosed forever.

Runtime-first. The profile is a **validator** (SHACL-shaped [16]) before it is a type. Type-level narrowing is a bonus applied on top, never the foundation — because edges arrive from adapters that do not know the profile, so the validator must exist regardless.

Trade-off. We give up compile-time guarantees that a `filesystem` node has exactly one parent (the API still returns `parents(): NodeId[]`) in exchange for a single code path that every profile shares. The validator is the SSOT.

Verification: `groups-core/tests/profiles.test.ts` — *"is the whole pitch: a filesystem and a tag cloud differ only by a dial"*, in which the identical two edges succeed under `flatTags` and fail under `filesystem`.

5.7 EdgeDelta as the sole write primitive

Choice. Every mutation — add, remove, re-parent, rename-in-place, reorder — is an `EdgeDelta { added, removed, upsertNodes }` (`groups-core/src/model.ts`), applied by `applyDelta()`. `invert()` produces the inverse.

Rationale. Undo becomes Command, not Memento — we never snapshot the space, we swap `added` and `removed`. Event-sourced adapters get a log for free; Zanzibar-style change-feed consumers get a stream for free [23]. `defineGroups().subscribe()` (`groups-core/src/define.ts`) exposes it.

Trade-off. Ergonomic wrappers (`add`, `remove`, `move`) must reconstruct their delta to record it, which is mild bookkeeping in `define.ts`. Worth it: one write path means one place to enforce every invariant.

5.8 Names and order on the edge, not the node

Answers §2.2(h).

Choice. `Edge.label` and `Edge.order` (`groups-core/src/model.ts`). Order is a fractional-index string (`groups-core/src/order.ts`).

Rationale. The Unix dentry insight — "names are not part of the inode but rather of the

dentry" [2] — and Git's tree entry [22]. It is what makes *the same item, in two groups, under two names, in two positions* representable at all. An item in three groups needs three ranks; a `position` column on the item cannot express that.

Fractional indexing [34] means a rank is a plain sortable string, so ordering works on **every backend with zero new capabilities**. ⚠️ **Sharp edge, documented in `order.ts`** :

`localeCompare` and locale-aware DB collations *silently corrupt* base-62 key order — the list comes out *mostly* right, which is what makes the bug expensive. Use binary comparison; on Postgres, `COLLATE "C"` .

5.9 Drag-and-drop: ADD is the default, MOVE needs a modifier

Answers §2.2(g).

Choice. `resolveDrop()` (`groups-ui/src/drag.ts`) returns

`{ operation: 'add' | 'move' | 'reorder', valid, reason, destructive, indicator }` .

Plain drag = **ADD a parent**. `⇧/Alt` = **MOVE**. `MEMBERSHIP_ACTIONS` exposes Gmail's two-verb menu for when there is room for two buttons.

Rationale — we invert Finder deliberately, on three grounds of increasing weight:

1. MOVE destroys an edge the user often **cannot see** (the source group may be off-screen, or the node may have five other parents).
2. MOVE is **undefined** when the drag starts in a search result or a flat list — there is no source group. ADD always has a meaning.
3. The undo costs are asymmetric: an accidental ADD is visible and one click to fix; an accidental MOVE silently removes something from a folder nobody was looking at.

Trade-off. We violate the muscle memory of every desktop file manager. We accept this because the failure modes are not symmetric, and because Gmail — the largest multi-membership system in the world — resolved the same ambiguity by refusing to have it (`Label` vs `Move to`).

Verification: `groups-ui/tests/drag.test.ts` , including *"falls back to ADD when there is no source group to move out of"*.

5.10 Counts as de-duplicated sets

Answers §2.2(c).

Choice. `countIn()` (`groups-core/src/closure.ts`) computes

`|{ item : g ∈ ancestors*(item) }|` — a **set**, always. `facetPanel()` (`groups-core/src/projections/facets.ts`) accumulates into a `Set` before taking `.size`, which makes double-counting *unrepresentable* rather than merely avoided.

Rationale. $\sum \text{children.count}$ is wrong the moment the structure is not a tree [6], and *wrong-but-plausible* numbers are the worst kind. Note the pleasant corollary from the search-engine literature: indexing a *set of group ids* (rather than a list of paths) makes correct distinct-document counting fall out of the postings-list structure at zero cost.

Known limitation, surfaced honestly. Correct *disjunctive* facet counts require **N+1 queries** [6, 29]. This belongs in a capability record, not discovered later.

5.11 Intensional ("smart") groups as first-class objects

Choice. `IntensionalGroup` (`groups-core/src/intensional.ts`) with built-ins `unfiled()` (Zotero's Unfiled Items [11]) and `multiHomed()` — a view that only *has meaning* under polyhierarchy. **Leaf-only in v1:** a rule may not select over another rule's output.

Rationale. Users do not experience a smart folder as a different *kind* of thing — it sits in the sidebar next to a real one. So it must be nameable, orderable, and appear in every projection. But lifting the leaf-only restriction means implementing recursive Datalog with stratified negation and a fixpoint evaluator [17], which is a research project, not a feature.

Trade-off. Deliberately the one place we chose "possible later" over "possible now", and we say so.

5.12 The headless boundary

Choice. `groups-core` and `groups-ui` emit **configuration objects only**.

`groups-ui/src/registry.ts` provides a capability-ranked renderer registry (`PRIORITY` bands), scored on **(surface, profile)**. `groups-ui-vanilla` is the ~150-line reference renderer.

Rationale, and one non-obvious consequence. Scoring on *profile* (not just surface) lets us encode a finding that would otherwise be lost: **the tree view survives polyhierarchy least well and costs the most to get right**, while Miller columns survive it *natively* — because **a column stack is a path** (`groups-core/src/projections/columns.ts`), so it never has to guess which of several parents you are viewing a node under. Mark Miller himself generalized the technique to directed graphs [35]. So `createVanillaRegistry()` scores columns *above* the tree for polyhierarchical profiles, and below it for `filesystem` — expressing a design judgement as a score rather than hard-coding a component.

This is also our answer to Google Drive's retreat (§2.2(i)): **the model is honest, and the default projection is familiar**. We do not render a naked DAG and hope.

No reactivity in the core. `GroupSpace.revision` is an O(1) memoization key; any host (Reselect, MobX, signals, Zustand) can derive from it. Reactivity is a *target*, not the model.

5.13 Members: reference or literal

Choice. `Member<V> = { kind: 'ref', id } | { kind: 'value', value }` with a pluggable `IdentityStrategy` (`groups-core/src/model.ts`).

Rationale. Storing `'cheese'` in a node table and then referencing it is clean but absurd; forcing a 40 MB document to be a literal is worse. Git's trick resolves it: content-address the literals [22]. With the default strategy `hash('cheese') === 'cheese'`, so the literal case costs nothing — **and the edge table stays uniform (always `NodeId` on both ends), so no graph algorithm ever branches on this**.

5.14 Traceability summary

Problem (§2/§3)	Choice (§5)	Module
Four schemes, one model	Unified node + profiles	<code>groups-core/src/profile.ts</code>
<code>groups_to_tags</code> sync	One relation, two indexes	<code>groups-core/src/space.ts</code>
Blind transitivity lies	<code>EdgeKindDef</code> + <code>compose</code>	<code>groups-core/src/model.ts</code> , <code>closure.ts</code>

Problem (§2/§3)	Choice (§5)	Module
Diamond / double counting	De-duplicated set counts	<code>groups-core/src/closure.ts</code> , <code>projections/facets.ts</code>
Update storm	Group-DAG-only closure	<code>groups-core/src/closure.ts</code>
A node is not a row	<code>PathNode[]</code> , dual keying	<code>groups-core/src/projections/tree.ts</code>
Invisible cycles	<code>Violation.path</code> , <code>canAddTo</code>	<code>groups-core/src/space.ts</code> , <code>groups-ui/src/drag.ts</code>
Cyclic imported data	<code>isRecursive</code> , memoized walks	<code>groups-core/src/projections/tree.ts</code>
Paths ≠ identifiers	<code>allPaths</code> (capped), <code>primaryPath</code>	<code>groups-core/src/projections/paths.ts</code>
Multi-parent is invisible	<code>otherLocations</code> , twins, a11y name	<code>projections/paths.ts</code> , <code>groups-ui/src/views.ts</code>
Move vs. add ambiguity	ADD default, <code>↗</code> = MOVE	<code>groups-ui/src/drag.ts</code>
Per-membership order	Fractional index on the edge	<code>groups-core/src/order.ts</code>
<code>remove</code> ≠ <code>delete</code>	<code>removeFrom</code> / <code>deleteNode</code> / <code>orphansOf</code>	<code>groups-core/src/space.ts</code>
Taxonomy drift (MeSH)	<code>redundantEdge</code> lint	<code>groups-core/src/lint.ts</code>
DAG is unteachable	Columns default for poly profiles	<code>groups-ui-vanilla/src/registry.ts</code>

6. Conclusion & Future Directions

6.1 The architectural posture

`zodal-groups` makes one bet: **that folders, tags, taxonomies, and facets are the same data model under different constraints, and that the hierarchy is a view rather than a fact**. Everything else follows mechanically — the reified edge, the two indexes, the per-kind closure, the path-keyed projection, the profile dials.

The bet is not novel in theory. Zotero ships it as a checkbox [11]; Unix has shipped it since 1971 [1]; [Are.na](#) ships it as a product [20]; Zanzibar ships it at Google scale [23]. What is genuinely absent from the field is a library that (a) **parameterizes the constraints** rather than baking one scheme in, (b) treats **every hierarchy as a computed projection** with correct identity semantics, and (c) is **honest about what it cannot do** — no cycles, no arbitrary "primary parent", no compile-time cardinality.

The position this creates is a narrow one, and deliberately so. `zodal-groups` is the **acyclic, containment-shaped special case** of a graph — and it is a special case *precisely because* that is what makes closure, counts, and breadcrumbs well-defined at all. A user who needs cycles and arbitrary edges is better served by `zodal-graphs`, and we say so rather than degrade.

6.2 What we are least sure of

Intellectual honesty requires naming these:

- **The conservative composition default** (§5.2) will under-infer in vocabularies that expect `is_a ◦ part_of` chains to close. We have chosen silence over confident error, but real thesaurus authors may disagree.
- **The group-DAG-only closure** (§5.3) rests on the size asymmetry between the taxonomy and the item set. A workload with a *million*-node taxonomy would invalidate the "just rebuild it" strategy, and the incremental-maintenance path (with `path_count` /DRed [24]) would have to be built.
- **ADD-as-default** (§5.9) is the choice most likely to generate user complaints, precisely because it contradicts the file manager. We believe the asymmetry of the failure modes justifies it; we would revise on evidence.

6.3 Future directions

1. **Store adapters.** `groups-store-postgres` (recursive CTE via RPC — PostgREST's grammar cannot express a subquery at all), `groups-store-dexie` (`multiEntry` indexes), `groups-store-fs` (a sidecar manifest — *not* symlinks, which make cycles the caller's problem).
 2. **Live intensional groups.** BeOS live queries and Spotlight smart folders auto-update; ours re-evaluate on read. Live invalidation is the natural next step, and Zanzibar's Leopard watch-stream [23] is the reference design.
 3. **Virtualized rendering at scale.** `PathNode[]` is already the correct shape for TanStack Virtual; the work is a windowing adapter plus a type-ahead that searches the *model* rather than the DOM (a known trap: the rows the user is typing at mostly do not exist in the DOM).
 4. **Space-filling visualization.** An icicle chart falls out of `PathNode[]` for free (depth $\rightarrow$ x, index $\rightarrow$ y). A *treemap* does not: **no correct space-filling treemap of a DAG exists**, and any implementation must first project to a tree and say so.
 5. **Per-user membership (folksonomy) at scale.** The reified edge already carries `meta.assertedBy`, so the `(tag, object, identity)` triple is expressible; no adapter yet optimizes for it.
 6. **Constraint-profile inference.** Given an existing corpus of edges, infer the tightest profile that validates it — a migration and auditing tool, and a way to tell a user what shape their data is *actually* in.
-

7. References

1. IEEE/The Open Group. *POSIX.1-2017* — `link()`. <https://pubs.opengroup.org/onlinepubs/9699919799/functions/link.html>
2. The Linux Kernel. *Overview of the Linux Virtual File System*. <https://www.kernel.org/doc/html/latest/filesystems/vfs.html>
3. Miles A, Bechhofer S, eds. *SKOS Simple Knowledge Organization System Reference*. W3C Recommendation, 18 August 2009. <https://www.w3.org/TR/skos-reference/>
4. Gene Ontology Consortium. *Ontology Relations*. <https://geneontology.org/docs/ontology-relations/>

5. National Information Standards Organization. *ANSI/NISO Z39.19-2005 (R2010): Guidelines for the Construction, Format, and Management of Monolingual Controlled Vocabularies*. <https://www.niso.org/publications/ansiniso-z3919-2005-r2010>
6. Apache Solr. *Faceting* (multivalued fields; `excludeTags`). <https://solr.apache.org/guide/solr/latest/query-guide/faceting.html>
7. U.S. National Library of Medicine. *MeSH Record Types and Tree Numbers*. https://www.nlm.nih.gov/mesh/intro_record_types.html
8. Perugini S. Symbolic links in the Open Directory Project. *Information Processing & Management*. 2008;44(2):910–930. <https://doi.org/10.1016/j.ipm.2007.07.009>
9. Wikipedia. *Wikipedia:Categorization — cycles*. <https://en.wikipedia.org/wiki/Wikipedia:Categorization>
10. Adobe. *Lightroom Classic — Photo collections*. <https://helpx.adobe.com/lightroom-classic/help/photo-collections.html>
11. Zotero. *Collections and Tags* (incl. *Show Items from Subcollections, Unfiled Items*). https://www.zotero.org/support/collections_and_tags
12. Google Workspace Updates. *Simplifying Google Drive's folder structure and sharing models*. 2020. <https://workspaceupdates.googleblog.com/2020/08/drive-folder-structure-sharing-updates.html>
13. Google. *Drive API — Single-parenting behavior*. <https://developers.google.com/workspace/drive/api/guides/ref-single-parent>
14. Hearst MA. Design Recommendations for Hierarchical Faceted Search Interfaces. *ACM SIGIR Workshop on Faceted Search*, 2006. <https://people.ischool.berkeley.edu/~hearst/papers/faceted-search-06.pdf>
15. Motik B, et al., eds. *OWL 2 Web Ontology Language Profiles*. W3C Recommendation. <https://www.w3.org/TR/owl2-profiles/>
16. Knublauch H, Kontokostas D, eds. *Shapes Constraint Language (SHACL)*. W3C Recommendation, 20 July 2017. <https://www.w3.org/TR/shacl/>
17. Abiteboul S, Hull R, Vianu V. *Foundations of Databases*. Addison-Wesley; 1995. (Extensional vs. intensional databases; Datalog.) <http://webdam.inria.fr/Alice/>
18. Google. *Gmail API — `users.labels` resource*. <https://developers.google.com/gmail/api/reference/rest/v1/users.labels>
19. Obsidian. *Tags — nested tags*. <https://help.obsidian.md/tags>
20. *Are.na*. *What is Are.na?* (blocks, channels, connections). <https://www.are.na/about>
21. Danbooru. *Help: Tag Implications*. https://danbooru.donmai.us/wiki_pages/help:tag_implications

22. Chacon S, Straub B. *Pro Git* — 10.2 Git Internals: Git Objects (blobs, trees, tree entries). <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>
23. Pang R, et al. Zanzibar: Google's Consistent, Global Authorization System. *USENIX ATC*, 2019. (Relation tuples; the Leopard transitive-closure index.) <https://www.usenix.org/conference/atc19/presentation/pang>
24. Gupta A, Mumick IS, Subrahmanian VS. Maintaining Views Incrementally. *ACM SIGMOD*, 1993. (The DRed — Delete and Rederive — algorithm.) <https://doi.org/10.1145/170035.170066>
25. PostgreSQL. *WITH Queries (Common Table Expressions)* — `WITH RECURSIVE` . <https://www.postgresql.org/docs/current/queries-with.html>
26. PostgreSQL. `ltree` — *hierarchical tree-like data type*. <https://www.postgresql.org/docs/current/ltree.html>
27. Celko J. *Joe Celko's Trees and Hierarchies in SQL for Smarties*. 2nd ed. Morgan Kaufmann; 2012. (The nested set model.)
28. Algolia. `hierarchicalMenu` widget and the `lvl0` / `lvl1` / `lvl2` attribute convention. <https://www.algolia.com/doc/api-reference/widgets/hierarchical-menu/js/>
29. Meilisearch. *Search API* — `facets` and facet distribution. <https://www.meilisearch.com/docs/reference/api/search>
30. Cognitect. *Datomic* — *Indexes* (EAVT, AEVT, AVET, VAET). <https://docs.datomic.com/pro/query/indexes.html>
31. Boost. *Boost.MultiIndex* — *Tutorial* ("indices act as views to the internal collection"); *Boost.Bimap*. https://www.boost.org/doc/libs/release/libs/multi_index/doc/tutorial/index.html
32. Fowler M. CQRS. <https://martinfowler.com/bliki/CQRS.html>
33. W3C. *Accessible Rich Internet Applications (WAI-ARIA) 1.2* — `aria-owns` ; and *ARIA Authoring Practices Guide* — *Tree View pattern*. <https://www.w3.org/TR/wai-aria-1.2/#aria-owns> · <https://www.w3.org/WAI/ARIA/apg/patterns/treeview/>
34. Wallace E. *Realtime Editing of Ordered Sequences* (fractional indexing). Figma, 2017. <https://www.figma.com/blog/realtime-editing-of-ordered-sequences/>
35. Wikipedia. *Miller columns*. https://en.wikipedia.org/wiki/Miller_columns
36. Gamma E, Helm R, Johnson R, Vlissides J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley; 1994. (Composite — intent explicitly states *tree*.)
37. Mädler C, et al. *qSKOS* — *Quality Assessment for SKOS Vocabularies*. <https://github.com/cmader/qSKOS>

38. Zod. *API — Recursive objects* (and the documented infinite-loop behaviour on cyclical data). <https://zod.dev/api>
39. Atlassian. *Pragmatic drag and drop*. <https://atlassian.design/components/pragmatic-drag-and-drop/>
40. Bostock M. *d3-hierarchy*. <https://d3js.org/d3-hierarchy>

Internal references

- `docs/research/_reconciliation.md` — the 24 numbered decisions (D1–D24) and the resolution of conflicts between the five reports. **The SSOT.**
- `docs/research/README.md` — index of the five research reports (~3,900 lines, ~264 sources).
- `docs/zodal-groups-concept.md` — the thesis, stated for a general audience.
- `docs/research_guide.md` — routing index: which document answers which question.